

cnimbus

Manual do Usuário

Construindo e Inicializando Imagens de VM Linux Mínimas e Distroless

“A mesma ideia de um Dockerfile: declare o que você quer, execute um comando.”

Cobre: `prepare, build-disk, run, keygen`
Backends QEMU, VirtualBox, VMware, Hyper-V
A referência completa de diretivas do Nimbusfile
Assinatura de pieces, assinatura EFI-stub & UKI
Boot em hardware físico (`HARDBOOT eth/wifi/eth+wifi`)
Exemplos práticos e compiláveis

Esta versão do documento corresponde à linha de desenvolvimento v1.0 do `cnimbus`.
Gerado como parte da própria documentação do projeto – veja `README.md` e `Tasks.md` na raiz do repositório para a fonte de verdade subjacente, continuamente validada, que este manual resume.

Termos técnicos, comandos, nomes de flags/diretivas e todo código permanecem em inglês, como no projeto original.

Contents

1	Introdução	3
1.1	O Que É o cnimbus	3
1.2	Filosofia de Design	3
1.3	Como Este Manual Está Organizado	4
2	Instalação	5
2.1	Requisitos	5
2.2	Compilando a Partir do Código-Fonte	5
2.3	Compilando Suas Primeiras Pieces	6
3	Começo Rápido	7
3.1	Passo 1: Um Servidor Mínimo	7
3.2	Passo 2: O Nimbusfile	7
3.3	Passo 3: Compilar	8
3.4	Passo 4: Inicializar e Alcançar	8
4	O Nimbusfile	10
4.1	Conceitos	10
4.2	Referência Completa de Diretivas	10
4.2.1	Identidade da Imagem e Alvo de Boot	10
4.2.2	Rede	11
4.2.3	Processo e Sistema de Arquivos	11
4.2.4	Armazenamento	12
4.2.5	Configuração ao Vivo — A Diretiva AGENT	12
4.2.6	Firewall e Ciclo de Vida	13
4.2.7	Assinatura	14
4.3	Regras de Precedência Que Vale Memorizar	14
5	Referência de Linha de Comando	15
5.1	cnimbus prepare	15
5.2	cnimbus build-disk	15
5.3	cnimbus run	16
5.4	cnimbus keygen	17
5.5	Outros Subcomandos	17
6	Backends de Hypervisor	18
6.1	Visão Geral	18
6.2	QEMU	18
6.3	VirtualBox	18

6.4	VMware	18
6.5	Hyper-V	19
7	Segurança: Assinatura e Secure Boot	20
7.1	Assinatura de Pieces (Integridade e Autenticidade)	20
7.2	Assinatura de Kernel EFI-Stub e UKI	20
8	Boot em Hardware Físico	22
8.1	A Diretiva HARDBOOT	22
8.2	HARDBOOT eth	22
8.3	HARDBOOT wifi	23
8.4	HARDBOOT eth+wifi	23
9	Exemplos Práticos	24
9.1	env-config — Passando Configuração em Tempo de Execução	24
9.2	volume-persistent-disk — Estado Que Sobrevive a um Reboot	24
9.3	agent-http-kv — Configuração ao Vivo Via HTTP Puro	24
9.4	agent-vboxguest-kv — Configuração ao Vivo Sem Rede no Guest	25
9.5	multi-service — Mais de Um Processo	25
9.6	static-ip-firewall — Rede Mais um Firewall	25
9.7	format-raw-disk — Imagens de Disco Estilo Nuvem	25
9.8	healthcheck-restart — Detectando um Processo Travado	25
9.9	stopgrace-graceful-shutdown — Dando Tempo Para Trabalho em Andamento Terminar	26
9.10	hardboot-eth — Ethernet Real, Boot USB Real	26
9.11	hello-http-baremetal-proxmox — Uma Imagem, Dois Destinos, IPv4 e IPv6	26
9.12	Mensagens de Boot em Todos os Consoles	28
9.13	Sinais de Controle do Proxmox: Signal Shutdown e Ctrl+Alt+Del	28
9.14	secureboot-uki — Assinando e Registrando um Certificado Real	28
10	Solução de Problemas e Limitações Conhecidas	29
10.1	Nenhum Shell em Lugar Nenhum	29
10.2	Imagens ISO Não São Isohybrid	29
10.3	Aceleração WHPX no Windows	29
10.4	VMware Player e vTPM	29
10.5	Arquivos Grandes em COPY/ADD e FORMAT iso	30
10.6	Códigos de Saída	30
A	Referência Rápida de Diretivas	31
B	Métricas Medidas	32

Chapter 1

Introdução

1.1 O Que É o cnimbus

cnimbus é uma ferramenta de linha de comando que constrói imagens de máquina virtual Linux mínimas e com propósito específico a partir de um único arquivo declarativo — um **Nimbusfile** — e as inicializa sob um hypervisor real. O modelo mental é deliberadamente o mesmo que o Docker popularizou: você declara o que quer (uma versão de kernel, uma configuração de rede, um ou mais processos para executar), e o **cnimbus** faz o resto: busca e compila um kernel, monta um sistema de arquivos raiz mínimo baseado em BusyBox, e monta uma imagem ISO ou disco raw inicializável.

Onde o **cnimbus** difere fortemente de um runtime de containers é no que ele produz. Um container compartilha o kernel do host; uma imagem **cnimbus** é seu próprio kernel mais um sistema de arquivos raiz minúsculo (tipicamente abaixo de 20 MiB), somente leitura, residente em RAM. Não há shell instalado por padrão, nenhum gerenciador de pacotes, e nenhum armazenamento persistente gravável a menos que você declare explicitamente um **VOLUME**. Cada imagem é construída para exatamente um propósito: executar o processo (ou o punhado de processos) que você declarou, e nada mais.

1.2 Filosofia de Design

Quatro decisões moldam todo o resto deste manual:

- **Distroless.** Não há um userland de propósito geral. O BusyBox fornece apenas o pequeno conjunto de applets que a sequência de boot e os **SERVICES** que você declarou realmente precisam (`busybox --install` conecta exatamente esses, não os cerca de 400 applets que uma build padrão do BusyBox traz).
- **Sem publicação, sem registro hospedado.** O **cnimbus** não hospeda nem distribui “pieces” pré-construídas (binários de kernel/BusyBox/iptables). Você constrói as suas próprias, uma vez, e as armazena em cache localmente ou em infraestrutura que você controla. Não existe um equivalente do Docker Hub para o **cnimbus**, por decisão explícita de produto.
- **Somente auto-construído.** Todo artefato que termina na imagem é ou compilado pelo **cnimbus prepare** a partir de fonte upstream (kernel.org, BusyBox, iptables, wpa_supplicant) com sua proveniência verificada (assinaturas PGP onde o upstream publica, hash fixado em

todo o resto), ou explicitamente copiado por você via `COPY`. Nada é buscado de terceiros não verificados em tempo de boot ou de build.

- **Comportamento real, validado por boot.** Todo mecanismo descrito neste manual — cada diretiva, cada backend, cada flag — foi exercitado contra uma máquina virtual real, de verdade inicializada, durante o desenvolvimento deste projeto, não apenas testado por unidade. Onde uma limitação real foi encontrada (e várias foram), ela é documentada como tal, em vez de disfarçada.

1.3 Como Este Manual Está Organizado

- O Capítulo 2 coloca o `cnimbus` compilado e suas primeiras peças prontas.
- O Capítulo 3 percorre o caminho mais curto possível de um diretório vazio até uma VM inicializada.
- O Capítulo 4 é a referência completa de diretivas do Nimbusfile — o coração deste manual.
- O Capítulo 5 documenta todo subcomando e flag.
- O Capítulo 6 cobre os quatro backends de hypervisor suportados e como alcançar um serviço rodando dentro do guest.
- O Capítulo 7 cobre a assinatura de peças e a assinatura de kernel EFI-stub / Secure Boot / UKI.
- O Capítulo 8 cobre o perfil de boot opt-in para hardware físico (bare-metal).
- O Capítulo 9 percorre cada exemplo enviado no diretório `examples/` do repositório, do início ao fim.
- O Capítulo 10 reúne limitações conhecidas e suas soluções alternativas.
- Os apêndices trazem uma referência rápida de uma página das diretivas e as métricas medidas de requisitos não-funcionais do projeto.

Chapter 2

Instalação

2.1 Requisitos

- **Docker** (Desktop no Windows/macOS, Engine no Linux) ou **Podman** — este é o único requisito real. É onde a compilação real de kernel/BusyBox/iptables acontece (`cnimbus prepare`); `cnimbus build-disk` e `cnimbus run` não precisam de Docker/Podman nenhum, uma vez que as peças já existam. Go *não* é um requisito: o próprio binário `cnimbus` pode ser compilado inteiramente dentro de um container Docker/Podman, sem nenhum Go instalado no host — veja `BUILD.md` na raiz do repositório.
- **Opcional** — só necessário para de fato inicializar uma imagem com `cnimbus run` (o objetivo central do projeto é montar a imagem; inicializá-la é um passo posterior e dispensável): um entre **QEMU**, **VirtualBox**, **VMware Workstation/Player**, ou **Hyper-V**.

2.2 Compilando a Partir do Código-Fonte

Com Go instalado no host:

```
git clone <este-repositorio>
cd cnimbus
go build -o cnimbus ./cmd/cnimbus
```

Sem Go instalado no host — apenas Docker (ou Podman), o único requisito real do projeto:

```
docker run --rm -v "$(pwd)":/src -w /src \
-e CGO_ENABLED=0 -e GOOS=linux -e GOARCH=amd64 \
golang:1.26 go build -o cnimbus ./cmd/cnimbus
```

Ambos os caminhos produzem um único binário `cnimbus` (ou `cnimbus.exe` no Windows) sem dependências. Veja `BUILD.md` na raiz do repositório para a matriz completa de compilação cruzada (Windows, Linux e macOS, em amd64, arm64 e — experimentalmente — riscv64 só para Linux), incluindo os comandos equivalentes para cada plataforma via Docker/Podman.

2.3 Compilando Suas Primeiras Pieces

`prepare` é o único subcomando que fala com o Docker. Ele baixa e compila um kernel (verificando sua assinatura PGP contra as chaves publicadas pelo próprio `kernel.org`, buscadas em tempo real via Web Key Directory), constrói um BusyBox correspondente, e constrói um `iptables` estático — juntos chamados de *pieces*.

```
./cnimbus prepare --out ./pieces
```

A primeira execução leva alguns minutos (uma compilação real de kernel). Toda execução posterior é instantânea se nada mudou, já que o Thunder (o pequeno programa em Go que conduz a build real dentro do container Docker) verifica um lockfile sha256 antes de decidir pular o trabalho. O resultado fica namespaced por arquitetura em `./pieces/amd64/` (ou `./pieces/arm64/`):

```
pieces/amd64/  
vmlinuz  
busybox  
busybox-manifest.tsv  
iptables  
pieces.json  
pieces.sha256
```

Tudo a partir daqui — todo exemplo neste manual — assume que esse diretório existe.

Chapter 3

Começo Rápido

Este capítulo constrói a menor imagem funcional possível: um servidor HTTP estático em Go, alcançável a partir do seu host, inicializado sob o QEMU.

3.1 Passo 1: Um Servidor Mínimo

```
mkdir my-first-vm && cd my-first-vm
cat > server.go << 'EOF'
package main

import ("fmt"; "log"; "net/http")

func main() {
    http.HandleFunc("/", func(w http.ResponseWriter, r *http.Request) {
        fmt.Fprintln(w, "hello from cnimbus")
    })
    log.Println("listening on :8080")
    log.Fatal(http.ListenAndServe(":8080", nil))
}
EOF
GOOS=linux GOARCH=amd64 CGO_ENABLED=0 go build -o helloserver server.go
```

`CGO_ENABLED=0` importa: a imagem não tem biblioteca C instalada, então um binário dinamicamente ligado falharia ao iniciar.

3.2 Passo 2: O Nimbusfile

Um Nimbusfile mínimo usando os valores padrão (`latest-stable`, `latest`) já é suficiente para o exemplo acima. Mas em builds reais e reproduzíveis — especialmente em CI, ou quando você precisa que o mesmo Nimbusfile produza bit a bit a mesma imagem meses depois — é melhor fixar versões exatas em vez de confiar em “a mais recente de hoje”. Os dois exemplos abaixo mostram ambos os estilos.

Nimbusfile mínimo (versões flutuantes):

```
KERNEL latest-stable
BUSYBOX latest
```



```
ARCH amd64
HOSTNAME my-first-vm
DHCP true

COPY ./helloserver /usr/bin/helloserver
ENTRYPOINT /usr/bin/helloserver
CMD :8080

FORMAT iso
```

Nimbusfile com versões reais fixadas (build reproduzível):

```
KERNEL 6.9.4
BUSYBOX 1.36.1
ARCH amd64
HOSTNAME my-first-vm-pinned
DHCP true

COPY ./helloserver /usr/bin/helloserver
ENTRYPOINT /usr/bin/helloserver
CMD :8080

FORMAT iso
```

Com `KERNEL`/`BUSYBOX` fixados em números de versão exatos (em vez de `latest-stable/latest`), o mesmo Nimbusfile compila exatamente o mesmo par `kernel+BusyBox` hoje e em qualquer data futura — o único jeito de garantir que uma imagem seja realmente reproduzível, já que “a versão mais recente do `kernel.org`” muda com o tempo por definição. Antes de testar que este segundo Nimbusfile realmente funciona, compile as peças fixadas e depois monte a imagem:

```
../cnimbus prepare --kernel 6.9.4 --busybox 1.36.1 --out ../pieces-pinned
../cnimbus build-disk -f Nimbusfile.pinned --pieces ../pieces-pinned
```

3.3 Passo 3: Compilar

```
../cnimbus build-disk --pieces ../pieces
```

Isso produz `my-first-vm.iso` no diretório atual — tipicamente com menos de 20 MiB.

3.4 Passo 4: Inicializar e Alcançar

```
../cnimbus run my-first-vm.iso
```

Por padrão isso inicializa sob o QEMU com um console serial e redireciona a porta 8080 do host para a porta 8080 do guest. De outro terminal:

```
curl http://127.0.0.1:8080/
# hello from cnimbus
```

Esse é o ciclo completo: declarar, compilar, inicializar, alcançar. Todo o restante deste manual trata das muitas coisas que você pode adicionar ao Nimbusfile, ou flags que você pode passar para mudar como a imagem é construída ou inicializada.

Chapter 4

O Nimbusfile

4.1 Conceitos

Um Nimbusfile é um arquivo de texto puro, uma diretiva por linha (linhas longas podem ser continuadas com uma barra invertida no final, como em um Dockerfile). Comentários começam com `#`. `cnimbus init` escreve um Nimbusfile inicial totalmente comentado no diretório atual — útil como referência viva ao lado deste capítulo.

Existem duas categorias de diretiva:

- **Lidas apenas pelo prepare** (KERNEL, BUSYBOX, VGA, HARDBOOT): afetam qual kernel é compilado, então precisam ser conhecidas *antes* das pieces existirem, não apenas antes da imagem ser montada.
- **Lidas apenas pelo build-disk** (todo o resto): afetam como a imagem é montada a partir de pieces já existentes, e podem mudar entre builds sem recompilar nada.

ARCH é lida por *ambos* os comandos, já que determina tanto o que o **prepare** compila quanto quais pieces namespaced-por-arquitetura o **build-disk** deve buscar.

Toda diretiva que também tem uma flag de linha de comando com o mesmo nome segue uma regra consistente: **a flag sempre vence** sobre o Nimbusfile. Isso permite que um único Nimbusfile sirva a múltiplas builds (uma matriz de CI construindo tanto `amd64` quanto `arm64` a partir do mesmo arquivo, por exemplo) sem editá-lo.

4.2 Referência Completa de Diretivas

4.2.1 Identidade da Imagem e Alvo de Boot

Diretiva	Significado
KERNEL <version>	latest-stable, latest-longterm, ou uma versão explícita (ex. 6.9.4). Lida apenas pelo prepare.

Diretiva	Significado
BUSYBOX <version>	Uma versão explícita, ou latest para o padrão do próprio cnimbus. Lida apenas pelo prepare .
ARCH <amd64 arm64>	Arquitetura de destino (padrão amd64). Lida tanto pelo prepare quanto pelo build-disk .
VGA <true false>	Habilita um console VGA/framebuffer real (padrão false — o console serial está sempre disponível independentemente). Também ativa um banner no console: assim que a rede sobe, todo endereço IPv4 (e IPv6, se houver) de cada interface é impresso na tela — a única forma de descobrir o endereço da imagem sem um shell para entrar. Lida apenas pelo prepare .
HARDBOOT <none eth wifi eth+wifi>	Perfil de boot em hardware físico (padrão none). Muda quais drivers de kernel são compilados — veja o Capítulo 8. Lida apenas pelo prepare .
FORMAT <iso raw vhd>	iso : uma imagem de disco óptico inicializável para uso em VM. raw : um disco GPT (ESP UEFI + partição raiz SquashFS), o formato para templates de disco em nuvem e boot USB/bare-metal. vhd : o mesmo layout raw envolto em um footer VHD Fixed real, pronto para o Hyper-V sem precisar de um passo run separado.

4.2.2 Rede

Diretiva	Significado
HOSTNAME <name>	Hostname da imagem.
DHCP <true false>	Sobe a eth0 via DHCP no boot.
IP <addr> <mask> <gw>	IP estático em vez de DHCP; vence o DHCP se ambos estiverem definidos.
DNS <addr>	Um nameserver explícito, sobrescrevendo o que o DHCP forneceu. Repetível.
NTP <server false>	Sincroniza o relógio no boot (padrão pool.ntp.org). Repetível; false desativa. Precisa de rede configurada.
WIFI <ssid>	Rede WiFi para associar. Requer HARDBOOT wifi ou HARDBOOT eth+wifi . Veja o Capítulo 8.
WIFIPSK <passphrase hex-key>	Credencial WPA2-PSK para WIFI. Requer HARDBOOT wifi ou HARDBOOT eth+wifi .
WIFICOUNTRY <code>	Domínio regulatório de duas letras (ex. US , BR) para o rádio WiFi. Requer HARDBOOT wifi ou HARDBOOT eth+wifi .

4.2.3 Processo e Sistema de Arquivos

Diretiva	Significado
USER <name>	Executa todo processo com esta conta sem privilégios (uid/gid 1000) em vez de root.
WORKDIR <path>	Diretório de trabalho para todo processo (padrão /).
ENV <KEY>=<VALUE>	Variável de ambiente exportada para todo processo. Repetível.
LABEL <KEY>=<VALUE>	Metadado livre escrito em <code>/etc/cnimbus-release</code> . Sem efeito no boot. Repetível.
EXPOSE <port>[/tcp /udp]	Documenta uma porta de escuta. Apenas informativo. Repetível.
ARG <NAME>[=<default>]	Variável de tempo de build, usável como <code>\${NAME}</code> em outros lugares. Sobrescreva com <code>--build-arg NAME=value</code> .
COPY [--chmod=<mode>] <src> <dest>	Arquivo/diretório/glob local para dentro da imagem. Deve corresponder ao ARCH.
ADD [--chmod=<mode>] <src> <dest>	Como COPY, mas <code>src</code> pode ser uma URL, e arquivos compactados são auto-extraídos.
ENTRYPOINT <cmd...>	O processo principal, reiniciado automaticamente. Forma shell ou exec.
CMD <args...>	Argumentos padrão anexados após ENTRYPOINT.
SERVICE <name> <cmd...>	Um processo adicional, supervisionado e reiniciado automaticamente. Repetível.

4.2.4 Armazenamento

Diretiva	Significado
VOLUME <device> <mount> [fstype] [required]	Monta um disco pré-existente e pré-formatado no boot. Nunca o formata. <code>fstype</code> é <code>vfat</code> (padrão) ou <code>ext4</code> . Sem <code>required</code> : uma montagem que falha é silenciosamente ignorada. Com <code>required</code> : uma montagem que falha interrompe o boot com uma mensagem FATAL clara. Repetível.
TMPSIZE <size>	Sobrescreve o <code>size=</code> do <code>tmpfs</code> para os diretórios executáveis em RAM (<code>bin</code> , <code>sbin</code> , <code>usr/bin</code> , <code>usr/sbin</code>); padrão 32m.

4.2.5 Configuração ao Vivo — A Diretiva AGENT

A família **AGENT** permite que uma VM em execução capte mudanças de configuração sem um rebuild ou reboot, escrevendo o valor buscado em `/var/run/cnimbus-kv.json` em um intervalo que seu **ENTRYPOINT** lê.

Forma	Canal
AGENT <url> [interval]	HTTP(S) puro, qualquer servidor. Funciona em qualquer hypervisor com rede no guest. Intervalo padrão 5s.
AGENT header <name>: <value>	Cabeçalho extra para a forma HTTP acima (end-points de metadados em nuvem frequentemente exigem um). Repetível.
AGENT vboxguest <property> [interval]	O canal real de Guest Properties do VirtualBox. Não precisa de rede no guest. Exclusivo do VirtualBox.
AGENT virtio-serial <device> [interval]	Um dispositivo de caractere virtio-serial do QEMU/Proxmox. Não precisa de qemu-guest-agent.
AGENT aws-imsd <path> [interval]	Serviço de metadados IMDSv2 do AWS EC2 (trata o handshake de PUT-token-depois-GET).
AGENT ibm-imsd <path> [interval]	Serviço de metadados do IBM Cloud VPC.
AGENT vmware <key> [interval]	guestinfo.<key> do VMware via o protocolo de I/O backdoor de baixa banda. Não precisa das VMware Tools. Só linux/amd64.

4.2.6 Firewall e Ciclo de Vida

Diretiva	Significado
FIREWALL <rule>	Uma linha de regra iptables (IPv4), aplicada no boot via um iptables estático construído pelo prepare. Repetível.
FIREWALL6 <rule>	O equivalente IPv6 de FIREWALL: mesma sintaxe de regra, aplicada via o mesmo binário estático (que também despacha como ip6tables), num conjunto de regras totalmente independente. Declarar um não afeta o outro. Repetível. Injeta automaticamente as mensagens ICMPv6 mínimas da RFC 4890 (Neighbor Discovery e erros de MTU) antes de qualquer regra sua — sem isso, uma política -P INPUT DROP bloquearia a própria resolução de endereço IPv6 (o equivalente do ARP), algo que FIREWALL nunca precisa porque o ARP não passa pelo iptables.
FIREWALL_ON_ERROR <open closed>	Comportamento de fallback se uma regra FIREWALL/FIREWALL6 falhar ao ser aplicada: open (padrão, aceita tudo) ou closed (bloqueia tudo exceto loopback/estabelecidas). Vale para os dois conjuntos de regras.

Diretiva	Significado
HEALTHCHECK [<code>--interval=<n></code>] [<code>--retries=<n></code>] <code><cmd...></code>	Executa <code><cmd...></code> periodicamente contra o ENTRYPOINT; mata/reinicia após falhas consecutivas. Padrões: intervalo 30s, tentativas 3.
RESTART <code><target></code> <code><always on-failure no></code>	Política de reinício para <code>entrypoint</code> ou um SERVICE nomeado.
STOPGRACE <code><seconds></code>	Quanto tempo um shutdown espera o ENTRYPOINT sair por conta própria após SIGTERM antes do SIGKILL. Padrão 10.

4.2.7 Assinatura

Diretiva	Significado
PIECESKEY <code><hex-pubkey></code>	Fixa a chave pública Ed25519 com a qual o <code>pieces.sha256</code> deve estar assinado. Veja o Capítulo 7.

4.3 Regras de Precedência Que Vale Memorizar

1. Uma flag de linha de comando sempre sobrescreve a diretiva de mesmo nome no Nimbusfile.
2. IP vence DHCP quando ambos estão presentes.
3. Um ENV posterior com a mesma chave sobrescreve um anterior.
4. `$$` é o escape para um `$` literal em qualquer argumento de diretiva (convenção do próprio Docker) — necessário para que ENTRYPOINT/CMD passem um `$VAR` literal adiante, para o shell expandir em *tempo de execução*, a partir de um valor ENV, em vez de em tempo de *parse* do Nimbusfile.

Chapter 5

Referência de Linha de Comando

5.1 cnimbus prepare

Compila as pieces (kernel, BusyBox, iptables estático) dentro do Docker.

Flag	Significado
-f <path>	Nimbusfile de onde ler os padrões de KERNEL/BUSYBOX/ARCH (padrão Nimbusfile).
--kernel <version>	Sobrescreve o KERNEL do Nimbusfile.
--busybox <version>	Sobrescreve o BUSYBOX do Nimbusfile.
--arch <amd64 arm64>	Sobrescreve o ARCH do Nimbusfile.
--out <dir>	Diretório onde escrever as pieces (padrão <code>./pieces</code> , namespaced por arquitetura).
--vga[=false]	Sobrescreve o VGA do Nimbusfile.
--hardboot	Sobrescreve o HARDBOOT do Nimbusfile. Veja o Capítulo 8.
<none eth wifi eth+wifi>	
--insecure-skip-key-verify	Pula a verificação PGP do tarball do kernel baixado. Não recomendado.
--pieces-sign-key <path>	Assina o <code>pieces.sha256</code> recém-escrito com esta chave Ed25519.
--jobs <n>	Paralelismo <code>make -j</code> para as builds de kernel/BusyBox/iptables (0 = auto-detecta a partir da memória disponível).

5.2 cnimbus build-disk

Monta uma imagem inicializável a partir de pieces já construídas. Nunca toca no Docker.

Flag	Significado
-f <path>	Nimbusfile a ler (padrão Nimbusfile).
-o <path>	Caminho da imagem de saída (padrão <code><hostname>.iso</code> , <code>.img</code> ou <code>.vhd</code>).

Flag	Significado
<code>--pieces <src></code>	Fonte de pieces pré-construídas: um diretório local ou um prefixo de URL <code>http(s)://</code> .
<code>--arch <amd64 arm64></code>	Sobrescreve o ARCH do Nimbusfile.
<code>--pieces-verify-key <hex></code>	Uma chave pública Ed25519 contra a qual o <code>pieces.sha256.sig</code> deve verificar.
<code>--pieces-cache-dir <dir></code>	Cache local para uma fonte <code>--pieces</code> remota.
<code>--no-pieces-cache</code>	Desativa completamente o cache de pieces.
<code>--pieces-insecure-http</code>	Permite uma fonte <code>--pieces http://</code> pura (recusada por padrão).
<code>--pieces-allow-unverified</code>	Permite uma fonte de pieces sem assinatura, sem manifesto de integridade nenhum.
<code>--no-lockfile</code>	Não escreve o <code><output>.lock</code> .
<code>--tmpdir <dir></code>	Diretório para os arquivos temporários de trabalho da build.
<code>--secureboot</code>	Assina o kernel EFI-stub enviado com uma implementação Authenticode em Go puro (sem Docker). Veja o Capítulo 7.
<code>--uki</code>	Monta e assina uma Unified Kernel Image (implica <code>--secureboot</code>).
<code>--secureboot-key <path></code>	Traz sua própria chave privada RSA para assinatura (deve vir junto com <code>--secureboot-cert</code>).
<code>--secureboot-cert <path></code>	Traz seu próprio certificado X.509 para assinatura.
<code>--secureboot-dir <dir></code>	Onde auto-gerar/reutilizar um par de chaves de assinatura quando nenhuma das opções acima é dada (padrão <code>./secureboot</code>).

5.3 *cnimbus* run

Inicializa uma imagem já construída sob um hypervisor real.

Flag	Significado
<code>--backend</code>	Hypervisor a usar (padrão <code>qemu</code>).
<code><qemu vbox vmware hyperv></code>	
<code>--arch <amd64 arm64></code>	Arquitetura da imagem (padrão <code>amd64</code>).
<code>--uefi</code>	Inicializa via UEFI/OVMF em vez de BIOS (só <code>amd64</code> — <code>arm64</code> é sempre UEFI).
<code>--ovmf-code <path></code>	/ Caminhos explícitos do firmware OVMF (auto-detectados caso contrário).
<code>--ovmf-vars <path></code>	
<code>--mem <MB></code>	RAM do guest (padrão 512).
<code>--smp <n></code>	Número de vCPUs do guest (só <code>--backend qemu</code>).
<code>--accel</code>	Backend de aceleração do QEMU (só <code>--backend qemu</code>).
<code><auto kvm hvf whpx tcg></code>	

Flag	Significado
<code>--hostfwd <host:guest></code>	Redirecionamento de porta TCP (padrão 8080:8080).
<code>--hostfwd-bind <addr></code>	Endereço do host ao qual o redirecionamento se vincula (padrão 127.0.0.1).
<code>--backend</code> <code>vbox vmware hyperv</code>	Veja o Capítulo 6 para notas específicas de cada backend.
<code>--vm-name <name></code>	Nome da VM a criar (removida após o desligamento a menos que <code>--vm-keep</code>).
<code>--vm-keep</code>	Não remove a VM após ela desligar.

5.4 *cnimbus* keygen

Gera as chaves de assinatura usadas em outros pontos deste manual.

```
cnimbus keygen --out my-signing-key.hex # assinatura de pieces (Ed25519)
cnimbus keygen --secureboot --out-dir ./secureboot # Secure Boot (RSA + X.509)
```

5.5 Outros Subcomandos

- `cnimbus init` — escreve um Nimbusfile inicial totalmente comentado no diretório atual.
- `cnimbus validate` — verifica um Nimbusfile (e a arquitetura de qualquer binário copiado via COPY) sem construir nada.
- `cnimbus kv-serve` — um pequeno servidor HTTP local para testar o mecanismo AGENT <url>, com TLS opcional e autenticação por bearer token.
- `cnimbus help` — lista todo subcomando e os códigos de saída documentados do projeto.

Chapter 6

Backends de Hypervisor

6.1 Visão Geral

`cnimbus run` unifica quatro backends sob uma única flag, `--backend`. A Tabela 6.1 resume o que cada um precisa e suporta.

Backend	Precisa	UEFI	FORMAT raw	Redir. porta
<code>qemu</code>	QEMU no PATH	Sim (OVMF)	Sim	<code>-netdev user</code>
<code>vbox</code>	VBoxManage	Sim	Sim	Redir. NAT
<code>vmware</code>	<code>vmrun</code> /VMware	Sim	Sim (envolto em VMDK)	Não automatizado
<code>hyperv</code>	Windows, admin	Sim (Ger. 2)	Só sim	Switch NAT

Table 6.1: Resumo de capacidades dos backends.

6.2 QEMU

O padrão. Nenhuma elevação necessária em qualquer plataforma.

```
cnimbus run my-image.iso
cnimbus run --uefi --accel kvm my-image.iso # Linux com KVM
```

6.3 VirtualBox

```
cnimbus run --backend vbox my-image.iso
cnimbus run --backend vbox --uefi my-image.iso
```

Nenhuma elevação é necessária em nenhuma plataforma testada por este projeto — o `VBoxManage` executa diretamente como o usuário que o invoca.

6.4 VMware

```
cnimbus run --backend vmware my-image.iso
```

O VMware Player (a versão gratuita) não pode usar vTPM — o Secure Boot funciona, mas a emulação de TPM exige o Workstation Pro ou o vSphere (criptografia da VM é um pré-requisito obrigatório que o Player não expõe). O redirecionamento de porta não é automatizado para este backend; use a própria configuração de rede do guest (um adaptador em bridge/NAT que você configura diretamente no VMware) para alcançar um serviço rodando dentro dele.

6.5 Hyper-V

```
cnimbus run --backend hyperv --uefi my-image.img
```

Exige FORMAT raw — o Hyper-V não tem um caminho de drive óptico virtual na implementação deste projeto, apenas um disco rígido virtual (um VHD Fixed, envolvendo a imagem raw de forma transparente). Exige uma sessão elevada (Administrador): execute a partir de uma janela do PowerShell aberta via “Executar como administrador”, ou aprove o prompt do UAC se o próprio *cnimbus* disparar um. Um guest alcançável via `--hostfwd` deve declarar um IP estático na faixa 192.168.200.0/24 — o switch NAT interno que este backend cria não tem servidor DHCP próprio.

Chapter 7

Segurança: Assinatura e Secure Boot

7.1 Assinatura de Pieces (Integridade e Autenticidade)

O `pieces.sha256` (escrito pelo `prepare`) é um manifesto de hashes: o `build-disk` verifica cada arquivo buscado contra ele, então um download corrompido ou uma substituição em trânsito é detectada. Isso é *integridade* — não prova *quem* publicou o manifesto.

```
cnimbus keygen --out my-signing-key.hex
cnimbus prepare --pieces-sign-key my-signing-key.hex
cnimbus build-disk --pieces-verify-key <a-chave-publica-impressa>
```

Com `--pieces-verify-key` (ou uma linha `PIECESKEY` no `Nimbusfile`) definida, o `build-disk` se recusa a construir a menos que a fonte de `pieces` tenha publicado um `pieces.sha256.sig` que verifique contra exatamente essa chave.

7.2 Assinatura de Kernel EFI-Stub e UKI

Assinar as `pieces` autentica as *entradas da build*. Isso não diz nada sobre o que o *firmware* confia em tempo de boot. `--secureboot` e `--uki` fecham essa lacuna: elas assinam a imagem EFI de boot real com uma implementação Authenticode em Go puro (sem Docker, sem ferramenta externa nenhuma — veja `internal/secureboot`), então um hypervisor com Secure Boot habilitado vai se recusar a executá-la a menos que seu certificado esteja registrado no seu banco de dados UEFI.

```
cnimbus build-disk --secureboot # assina o kernel diretamente
cnimbus build-disk --uki # monta + assina uma UKI completa
```

Sem flags explícitas de chave/certificado, uma identidade de assinatura (RSA-3072 + X.509 autoassinado) é gerada uma vez sob `--secureboot-dir` (padrão `./secureboot`) e reutilizada em toda build posterior — nunca regenerada silenciosamente. Veja o passo a passo de `examples/secureboot-uki/` no Capítulo 9 para os comandos exatos, testados de ponta a ponta, para registrar esse certificado na variável `db` UEFI de uma VM VirtualBox real e inicializar com o Secure Boot habilitado.

Uma limitação de plataforma confirmada: no momento em que este manual foi escrito, o módulo PowerShell do Hyper-V não expõe nenhum cmdlet para registrar um certificado fornecido pelo usuário — `Set-VMFirmware -SecureBootTemplate` só aceita os templates fixos da própria

Microsoft. O VirtualBox é o backend a usar para uma cadeia real de Secure Boot autoassinada hoje.

Chapter 8

Boot em Hardware Físico

8.1 A Diretiva HARDBOOT

Por padrão, as imagens do cnimbus são construídas apenas para hardware virtual: `virtio-net`, `virtio-blk`, e dispositivos paravirtualizados similares que nenhuma máquina física realmente tem. HARDBOOT é uma diretiva opt-in, inerte por padrão, que troca por drivers de hardware real em vez disso.

```
HARDBOOT eth # ou: wifi, ou eth+wifi para ambos
```

Isso também precisa ser passado para o `prepare` (muda qual kernel é compilado):

```
cnimbus prepare --hardboot eth
```

Um Nimbusfile cujo HARDBOOT não corresponde às peças contra as quais está sendo construído é rejeitado em tempo de `build-disk`, da mesma forma que um ARCH incompatível já é.

Cada valor isolado é exclusivo à sua própria família de drivers: HARDBOOT `eth` nunca constrói a pilha WiFi, e HARDBOOT `wifi` nunca constrói drivers de chipset Ethernet. HARDBOOT `eth+wifi` é o único valor que constrói ambos, para uma máquina física que tem tanto uma NIC cabeada quanto um rádio WiFi.

8.2 HARDBOOT eth

Adiciona drivers reais de chipset Ethernet das famílias Intel e1000/e1000e e Realtek R8169. FORMAT `raw` é exigido para um boot real via pendrive USB ou bare-metal (uma ISO simples não tem código de boot MBR, então não pode ser escrita em um pendrive USB e inicializada da forma que uma máquina real espera).

Como uma verificação prévia barata antes de gastar uma tentativa de boot físico: a NIC emulada *padrão* do QEMU e do VirtualBox é um chip real da família e1000, então exatamente a mesma pilha de drivers pode ser testada virtualmente primeiro (`cnimbus run --backend qemu --format raw` ou `--backend vbox`). Isso não exercita o caminho real de enumeração USB/MBR, mas pega de graça uma combinação obviamente quebrada de driver/kconfig.

8.3 HARDBOOT wifi

Adiciona um conjunto curado de drivers de chipset WiFi (liderado pelo Atheros AR9271, `ath9k_htc`), os blobs de firmware correspondentes (fixados por hash e embutidos no `initramfs` de boot), e um `wpa_supplicant` estaticamente ligado (o BusyBox não traz nenhum). Use as diretivas `WIFI` / `WIFIPSK` / `WIFICOUNTRY` do Capítulo 4 para configurar a rede.

Limitação honesta: não há hardware de rádio WiFi, nem emulação de chipset WiFi, disponível no QEMU ou VirtualBox — diferente da Ethernet cabeada, a prova de associação em hardware real deste perfil genuinamente exige um adaptador WiFi físico. Tudo até esse ponto (compilação do kernel, empacotamento de firmware, geração de configuração, tratamento de credenciais) é verificado hoje por inspeção direta de uma imagem real construída.

8.4 HARDBOOT eth+wifi

Constrói ambas as famílias de drivers no mesmo kernel e sobe ambas as interfaces no boot (`eth0` via DHCP/IP como usual, `wlan0` via `WIFI`/`WIFIPSK`/`WIFICOUNTRY`) — para uma máquina física que genuinamente tem tanto uma porta cabeada quanto um rádio WiFi. Exige as mesmas diretivas `WIFI`/`WIFIPSK`/`WIFICOUNTRY` que `HARDBOOT wifi` isoladamente exige; as mesmas duas lacunas de hardware real acima (boot USB/BIOS, associação WiFi) se aplicam.

```
HARDBOOT eth+wifi
WIFI MyNetwork
WIFIPSK correcthorsebattery
WIFICOUNTRY BR
DHCP true
```


Chapter 9

Exemplos Práticos

Cada exemplo abaixo vive em `examples/` no repositório e é totalmente autocontido: seu próprio Nimbusfile, seu próprio `README.md` com comandos exatos de build/boot, e (quando um binário complementar é necessário) seu próprio código-fonte pequeno em Go. Todos eles assumem que o `cnimbus` está compilado e que o `cnimbus prepare` já produziu `./pieces` para o ARCH de destino.

9.1 env-config — Passando Configuração em Tempo de Execução

Mostra **ENV**: valores de configuração embutidos em tempo de build, lidos pelo entrypoint como variáveis de ambiente comuns. O exemplo mais simples possível da filosofia “declare, não script”.

9.2 volume-persistent-disk — Estado Que Sobrevive a um Reboot

Mostra **VOLUME**: montando um disco pré-formatado e pré-anexado, de forma que um arquivo de log (ou qualquer outro estado) escrito durante um boot ainda esteja lá após um ciclo completo de energia. Todo o resto em uma imagem `cnimbus` é apenas-RAM por design — esta é a única válvula de escape deliberada.

```
VOLUME /dev/vda /mnt/data
ENTRYPOINT ["/bin/sh", "-c",
  "echo \"booted at $(date)\" >> /mnt/data/log.txt; sleep 3600"]
```

9.3 agent-http-kv — Configuração ao Vivo Via HTTP Puro

Mostra **AGENT** `<url>`: uma VM em execução consultando um endpoint HTTP do lado do host a cada poucos segundos e escrevendo a resposta em `/var/run/cnimbus-kv.json`, permitindo que sua própria aplicação capte mudanças de configuração sem um rebuild ou reboot. Funciona em qualquer hypervisor com rede no guest.

9.4 agent-vboxguest-kv — Configuração ao Vivo Sem Rede no Guest

A mesma ideia acima, mas pelo canal real de Guest Properties do VirtualBox em vez de HTTP — útil quando o guest deliberadamente não tem acesso à rede nenhum, e a configuração ainda precisa chegar até ele.

9.5 multi-service — Mais de Um Processo

Mostra **SERVICE**: executando um processo adicional, supervisionado e reiniciado automaticamente, ao lado do **ENTRYPOINT**, cada um gerenciado independentemente.

9.6 static-ip-firewall — Rede Mais um Firewall

Mostra **IP** (um endereço estático, demonstrando que ele vence o DHCP quando ambos estão definidos) e **FIREWALL**: aplicando regras reais de **iptables** no boot.

```
IP 192.168.56.10 255.255.255.0 192.168.56.1
FIREWALL -P INPUT DROP
FIREWALL -A INPUT -p tcp --dport 8080 -j ACCEPT
```

Tráfego de loopback e conexões estabelecidas/relacionadas são sempre aceitas automaticamente antes de qualquer regra **FIREWALL** rodar — você nunca precisa declarar nenhuma das duas você mesmo.

9.7 format-raw-disk — Imagens de Disco Estilo Nuvem

Mostra **FORMAT raw**: produzindo um disco raw particionado em GPT em vez de uma ISO, o formato que plataformas de nuvem (Proxmox, e provisionamento baseado em imagem de disco similares) esperam, e o mesmo formato que **HARDBOOT** exige para um boot físico.

9.8 healthcheck-restart — Detectando um Processo Travado

Mostra **HEALTHCHECK** e **RESTART**: um servidor deliberadamente instável que para de responder (sem travar) após 20 segundos, pego e forçado a reiniciar por três verificações de saúde consecutivas falhas.

```
HEALTHCHECK --interval=5 --retries=3 wget -q -O /dev/null http://127.0.0.1:8080/health
RESTART entrypoint always
```

O console mostra a sequência exata de escalonamento: três verificações falhas, depois **SIGTERM**, depois (se ainda estiver vivo) **SIGKILL** (código de saída 137 = 128+9, confirmando que foi mesmo um kill), depois um reinício limpo.

9.9 stopgrace-graceful-shutdown — Dando Tempo Para Trabalho em Andamento Terminar

Mostra STOPGRACE: um servidor que termina uma “transação” em andamento de 8 segundos após receber SIGTERM, em vez de morrer imediatamente — o cenário para o qual o orçamento padrão de cerca de 1 segundo de shutdown do init do BusyBox é muito curto.

```
STOPGRACE 15
```

9.10 hardboot-eth — Ethernet Real, Boot USB Real

Mostra HARDBOOT eth de ponta a ponta: o passo `prepare --hardboot eth`, uma verificação prévia virtual com e1000/e1000e sob o QEMU e o VirtualBox, e os passos exatos de dd/Rufus para escrever a imagem raw resultante em um pendrive USB para um boot físico real.

9.11 hello-http-baremetal-proxmox — Uma Imagem, Dois Destinos, IPv4 e IPv6

Um único Nimbusfile que sobe sem modificação em hardware físico real (via pendrive UEFI) e dentro do Proxmox (como o CD-ROM virtual de uma VM), alcançável por qualquer lugar da internet tanto em IPv4 quanto em IPv6:

```
KERNEL latest-stable
BUSYBOX latest
ARCH amd64
HOSTNAME cnimbus-hello-http-demo

WIFI cnimbus-placeholder-ssid
WIFIPSK cnimbus-placeholder-psk
WIFICOUNTRY US
HARDBOOT eth+wifi

VGA true
DHCP true

COPY ./helloserver /usr/bin/helloserver
ENTRYPOINT /usr/bin/helloserver
CMD :8080

FIREWALL -P INPUT DROP
FIREWALL -A INPUT -p tcp -s 0.0.0.0/0 --dport 8080 -j ACCEPT
FIREWALL6 -P INPUT DROP
FIREWALL6 -A INPUT -p tcp -s ::/0 --dport 8080 -j ACCEPT

FORMAT iso
```

HARDBOOT eth+wifi é aditivo sobre os drivers de VM que `vm-amd64.fragment` já traz (e1000/virtio-net) — é exatamente isso que permite uma única imagem inicializar tanto em hardware real quanto dentro do Proxmox sem mudança nenhuma: a NIC virtual do Proxmox já é coberta pelos drivers de

VM, independentemente do HARDBOOT; a NIC (ou placa WiFi) de uma máquina real é coberta pelos drivers que o HARDBOOT adiciona por cima. WIFI/WIFIPSK/WIFICOUNTRY são obrigatórios sempre que HARDBOOT inclui `wifi` — os valores acima são apenas placeholders (este exemplo só exercita Ethernet cabeada nos dois destinos); substitua-os por credenciais reais se o hardware físico de destino precisar associar via WiFi.

VGA true ativa o banner de IP do console: assim que a rede sobe, a tela (o monitor físico, ou o console noVNC/SPICE do Proxmox) mostra algo como:

```
cnimbus: IPv4 address: 10.0.2.15 (eth0)
cnimbus: IPv6 address: fd00::... (eth0)
```

FIREWALL/FIREWALL6 (seção “Referência Completa de Diretivas”) são dois conjuntos de regras independentes — declarar um não afeta o outro. Ambos ficam abertos a `0.0.0.0/0/::/0` na porta 8080 aqui, exatamente como pedido; loopback e conexões já estabelecidas são sempre aceitas automaticamente antes de qualquer regra rodar (mesmo comportamento do exemplo `static-ip-firewall` anterior).

Verificado de ponta a ponta em hardware físico real e num Proxmox real: o kernel sobe, o DHCP obtém um lease em `eth0`, o banner de IP imprime tanto o endereço IPv4 quanto o IPv6, um `curl` de outra máquina na mesma rede (inclusive `curl -6`) retorna a resposta HTTP real, os dois conjuntos de FIREWALL/FIREWALL6 se aplicam sem nenhuma falha de regra, e os dois sinais de controle do Proxmox (“Signal Shutdown” via ACPI e Ctrl+Alt+Del pelo console) desligam/reiniciam a VM de forma correta.

Boot em hardware real (apenas UEFI — esta ISO não é isohybrid, veja “Imagens ISO Não São Isohybrid”):

```
# Linux/macOS
sudo dd if=cnimbus-hello-http-demo.iso of=/dev/sdX bs=4M \
    status=progress conv=fsync
# Windows: Rufus, modo "DD Image", apontado para o .iso
```

Um pendrive gravado desta forma inicializa em qualquer hardware UEFI com um disco USB anexado como um disco SATA/SCSI comum — HARDBOOT `eth/eth+wifi` traz o driver de armazenamento USB necessário para o kernel redescobrir o próprio pendrive depois que o firmware UEFI entrega o controle a ele. Uma ferramenta de pendrive multiboot (Ventoy e equivalentes, que inicializam encadeando o GRUB contra um arquivo `.iso` numa partição FAT/exFAT em vez de gravar a ISO diretamente no disco) também funciona sem configuração adicional: a imagem varre toda partição FAT/exFAT do pendrive procurando qualquer arquivo `*.iso` e o monta via loop.

Se o pendrive carrega mais de uma imagem cnimbus, o console mostra qual delas realmente subiu — lido de um `CNIMBUS.CFG` em texto puro na raiz da própria ISO:

```
cnimbus: identified boot image:
HOSTNAME=cnimbus-hello-http-demo
ARCH=amd64
FORMAT=iso
CNIMBUS_VERSION=...
```

Boot no Proxmox: envie a ISO para um storage que aceite imagens ISO, crie uma VM sem disco algum (a imagem inteira roda em RAM), anexe a ISO como unidade de CD/DVD, defina a ordem de boot para inicializar a partir dela, e inicie a VM.

9.12 Mensagens de Boot em Todos os Consoles

Com `--vga/VGA true`, toda mensagem que a imagem imprime no boot (o banner de IP, o estado dos serviços) aparece tanto no monitor físico/console de vídeo quanto na porta serial, independente de qual deles a linha de comando do kernel declara como `/dev/console` principal. Isso importa porque o kernel sempre imprime seu próprio `dmesg` em todos os consoles declarados, mas o userspace por padrão escreveria apenas no último deles — uma tela de monitor que mostra o `dmesg` do kernel e nada da própria imagem não significa que o boot travou, apenas que a saída está indo só para a serial.

9.13 Sinais de Controle do Proxmox: Signal Shutdown e Ctrl+Alt+Del

Esta imagem responde aos dois sinais de controle que o Proxmox oferece pela interface web:

- “**Signal Shutdown**” envia um pedido de desligamento ACPI; a imagem completa o desligamento gracioso (para os serviços, desmonta o que precisa ser desmontado, e desliga a VM) igual a rodar `poweroff` localmente teria.
- **Ctrl+Alt+Del** pelo console reinicia a VM através do mesmo ciclo gracioso de `RESTART/STOPGRACE`.

Os dois foram verificados contra um Proxmox real e contra um Hyper-V Generation 1 real.

9.14 `secureboot-uki` — Assinando e Registrando um Certificado Real

Mostra `--secureboot/--uki` de ponta a ponta, incluindo os comandos exatos e testados de `VBoxManage modifynvram` para registrar um certificado real, autogerado, na variável `db` UEFI de uma VM VirtualBox (usando o `cert-to-efi-sig-list` do `efitools` para construir uma `EFI_SIGNATURE_LIST` real) e inicializar com o Secure Boot habilitado — o primeiro boot real verificado por Secure Boot de um kernel assinado pelo cnimbus documentado neste projeto, sem usar nenhuma chave de demonstração de fornecedor.

Chapter 10

Solução de Problemas e Limitações Conhecidas

10.1 Nenhum Shell em Lugar Nenhum

Não há shell instalado na imagem por design (além do que a própria invocação em forma shell do **ENTRYPOINT** usa internamente). Você não pode fazer **ssh** ou login no console de uma VM cnimbus em execução para explorar interativamente — se você precisa de visibilidade em tempo de execução, construa isso dentro do seu próprio **ENTRYPOINT**, ou use **AGENT** para configuração ao vivo e um log apoiado em **VOLUME** para saída persistida.

10.2 Imagens ISO Não São Isohybrid

Uma imagem **FORMAT iso** simples tem entradas duplas de catálogo de boot El Torito (BIOS e UEFI em amd64) mas nenhum código de boot MBR no byte 0. Ela vai inicializar corretamente na unidade óptica virtual de qualquer hypervisor suportado, mas escrevê-la diretamente em um pendrive USB e esperar que uma máquina real inicialize a partir dela não vai funcionar. Use **FORMAT raw** para boot USB/bare-metal em vez disso — esta é uma escolha de design explícita e documentada, não um descuido.

10.3 Aceleração WHPX no Windows

`--accel auto` / `--accel whpx` foi observado falhando completamente em pelo menos um host Windows real com QEMU: **WHPX: Unexpected VP exit code 4**. `--accel tcg` (emulação por software) é um fallback confiável — mais lento, mas inicializa.

10.4 VMware Player e vTPM

O VMware Player (a versão gratuita) não consegue ligar uma VM com `vtpm.present = "TRUE"` — exige que a VM esteja criptografada primeiro, um recurso exclusivo do Workstation Pro/vSphere. Use o VirtualBox ou o Hyper-V se seu fluxo de trabalho precisa de um TPM virtual.

10.5 Arquivos Grandes em COPY/ADD e FORMAT iso

FORMAT iso tem um limite rígido em cima do **TMPSIZE**: a carga combinada de kernel + **initramfs** do estágio 1 é carregada via a entrada de boot “sem emulação” do El Torito, cujo campo de tamanho tem 16 bits de unidades de 512 bytes (aproximadamente um orçamento prático de 24 MiB para o **initramfs** comprimido). Um **COPY/ADD** em um dos quatro diretórios executáveis em RAM grande o suficiente para exceder isso faz o **build-disk** falhar com uma mensagem nomeando o arquivo culpado. **FORMAT raw** não tem esse limite — troque para ele para qualquer coisa grande.

10.6 Códigos de Saída

O **cnimbus** diferencia modos de falha via código de saída em vez de sempre sair com 1:

- **1** — uma falha genérica/não classificada.
- **4** — uma falha de integridade ou verificação de assinatura das peças.
- **5** — uma busca upstream (kernel.org, BusyBox, etc.) não pôde ser alcançada ou resolvida.

Execute **cnimbus help** para a lista autoritativa e atualizada.

Appendix A

Referência Rápida de Diretivas

Categoria	Diretivas
Alvo de boot	KERNEL, BUSYBOX, ARCH, VGA, HARDBOOT, FORMAT
Rede	HOSTNAME, DHCP, IP, DNS, NTP, WIFI, WIFIPSK, WIFICOUNTRY
Processo/SA	USER, WORKDIR, ENV, LABEL, EXPOSE, ARG, COPY, ADD, ENTRYPOINT, CMD, SERVICE
Armazenamento	VOLUME, TMPsize
Config. ao vivo	AGENT (http, vboxguest, virtio-serial, aws-imsd, ibm-imsd, vmware)
Firewall/ciclo de vida	FIREWALL, FIREWALL_ON_ERROR, HEALTHCHECK, RESTART, STOPGRACE
Assinatura	PIECESKEY

Appendix B

Métricas Medidas

As métricas de requisitos não-funcionais a seguir foram medidas contra um `prepare + build-disk` real, e (onde indicado) um boot real em QEMU, na própria máquina de desenvolvimento deste projeto — veja `REQUIREMENTS.md` para a tabela completa e a metodologia.

Métrica	Medido	Orçamento
Tamanho da imagem (exemplo mínimo)	19,11 MiB	32 MiB
Tamanho do <code>vmlinuz</code>	3,31 MiB	12 MiB
RSS de pico do <code>build-disk</code>	36,03 MiB	512 MiB
Tempo de parede do <code>build-disk</code> (pieces em cache)	0,53 s	30 s
Boot até o entrypt (emulação por software)	6,79 s	—

Table B.1: Métricas medidas de NFR.

Este manual resume o próprio `README.md`, `BUILD.md`, `Tasks.md`, e `REQUIREMENTS.md` do projeto no momento em que foi escrito. Esses arquivos, e os logs reais de boot referenciados ao longo do histórico de commits deste projeto, permanecem a fonte de verdade autoritativa e continuamente atualizada.